

Assignment 2

[Start Assignment](#)

- Due Saturday by 11:59p.m.
- Points 15
- Submitting a file upload
- File Types pdf, zip, and z
- Available Mar 19 at 12a.m. - Mar 29 at 11:59p.m.

Acknowledgement:

This project is adapted form <http://ai.berkeley.edu> (<http://ai.berkeley.edu/>)

Academic Honesty:

- Any sign of academic misconduct will be followed up and can have a serious academic penalty. It is the responsibility of students to be aware of the actions that constitute academic misconduct. We will be checking your code against other submissions in the class for logical redundancy. If you copy someone else's code and submit it with minor changes, we will know. The code should be only based on your efforts. Please see:

http://calendar.uoit.ca/content.php?catoid=22&navoid=879#Academic_conduct
(http://calendar.uoit.ca/content.php?catoid=22&navoid=879#Academic_conduct)

Introduction

In this assignment, you will implement value iteration and Q-learning. You will test your agents first on Gridworld (from class), then apply them to a simulated robot controller (Crawler).

As in previous assignment, this assignment includes an autograder for you to test your solutions on your machine (your grade is not solely based on autograder) . This can be run on all questions with the command:

```
python autograder.py
```

It can be run for one particular question, such as q2, by:

```
python autograder.py -q q2
```

It can be run for one particular test by commands of the form:

```
python autograder.py -t test_cases/q2/3-bridge
```

The code for this assignment contains the following files, available as [here](https://learn.ontariotechu.ca/courses/31258/files/4998463?wrap=1)

(<https://learn.ontariotechu.ca/courses/31258/files/4998463?wrap=1>)_ ↓

(https://learn.ontariotechu.ca/courses/31258/files/4998463/download?download_frd=1)

Files you'll edit:

<code>valueIterationAgents.py</code>	A value iteration agent for solving known MDPs.
<code>qlearningAgents.py</code>	Q-learning agents for Gridworld, Crawler and Pacman.

Files you might want to look at:

<code>mdp.py</code>	Defines methods on general MDPs.
<code>learningAgents.py</code>	Defines the base classes <code>ValueEstimationAgent</code> and <code>QLearningAgent</code> , which your agents will extend.
<code>util.py</code>	Utilities, including <code>util.Counter</code> , which is particularly useful for Q-learners.
<code>gridworld.py</code>	The Gridworld implementation.
<code>featureExtractors.py</code>	Classes for extracting features on (state, action) pairs. Used for the approximate Q-learning agent (in <code>qlearningAgents.py</code>).

Files to Edit and Submit: You will fill in portions of `valueIterationAgents.py`, `qlearningAgents.py`, during the assignment. Please do not change the other files in this distribution or submit any of our original files other than these files.

MDPs

To get started, run Gridworld in manual control mode, which uses the arrow keys:

```
python gridworld.py -m
```

You will see the two-exit layout from class. The blue dot is the agent. Note that when you press up, the agent only actually moves north 80% of the time. Such is the life of a Gridworld agent!

You can control many aspects of the simulation. A full list of options is available by running:

```
python gridworld.py -h
```

The default agent moves randomly

```
python gridworld.py -g MazeGrid
```

You should see the random agent bounce around the grid until it happens upon an exit. Not the finest hour for an AI agent.

Note: The Gridworld MDP is such that you first must enter a pre-terminal state (the double boxes shown in the GUI) and then take the special 'exit' action before the episode actually ends (in the true terminal state called `TERMINAL_STATE`, which is not shown in the GUI). If you run an episode manually, your total return may be less than you expected, due to the discount rate (`-d` to change; 0.9 by default).

Look at the console output that accompanies the graphical output (or use `-t` for all text). You will be told about each transition the agent experiences (to turn this off, use `-q`).

As in Pacman, positions are represented by `(x, y)` Cartesian coordinates and any arrays are indexed by `[x][y]`, with `'north'` being the direction of increasing `y`, etc. By default, most transitions will receive a reward of zero, though you can change this with the living reward option (`-r`).

Question 1 (7 points): Value Iteration

Recall the value iteration state update equation:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

Write a value iteration agent in `ValueIterationAgent`, which has been partially specified for you in `valueIterationAgents.py`. Your value iteration agent is an offline planner, not a reinforcement learning agent, and so the relevant training option is the number of

iterations of value iteration it should run (option `-i`) in its initial planning phase.

`ValueIterationAgent` takes an MDP on construction and runs value iteration for the specified number of iterations before the constructor returns (it put the values calculated in each iteration in `self.values`).

Value iteration computes k -step estimates of the optimal values, V_k . In addition to

`runValueIteration`, implement the following methods for `ValueIterationAgent` using V_k :

- `computeActionFromValues(state)` computes the best action according to the value function given by `self.values`.
- `computeQValueFromValues(state, action)` returns the Q-value of the (state, action) pair given by the value function given by `self.values`.

These quantities are all displayed in the GUI: values are numbers in squares, Q-values are numbers in square quarters, and policies are arrows out from each square.

Important: Use the “batch” version of value iteration where each vector V_k is computed from a fixed vector V_{k-1} (like in lecture), not the “online” version where one single weight vector is updated in place. This means that when a state’s value is updated in iteration k based on the values of its successor states, the successor state values used in the value update computation should be those from iteration $k - 1$ (even if some of the successor states had already been updated in iteration k).

Note: Some useful mdp methods that you will use are:

- `mdp.getStates()`
- `mdp.getPossibleActions(state)`
- `mdp.getTransitionStatesAndProbs(state, action)`
- `mdp.getReward(state, action, nextState)`
- `mdp.isTerminal(state)`

Hint: You may optionally use the `util.Counter` class in `util.py`, which is a dictionary with a default value of zero.

Note: Make sure to handle the case when a state has no available actions in an MDP (think about what this means for future rewards).

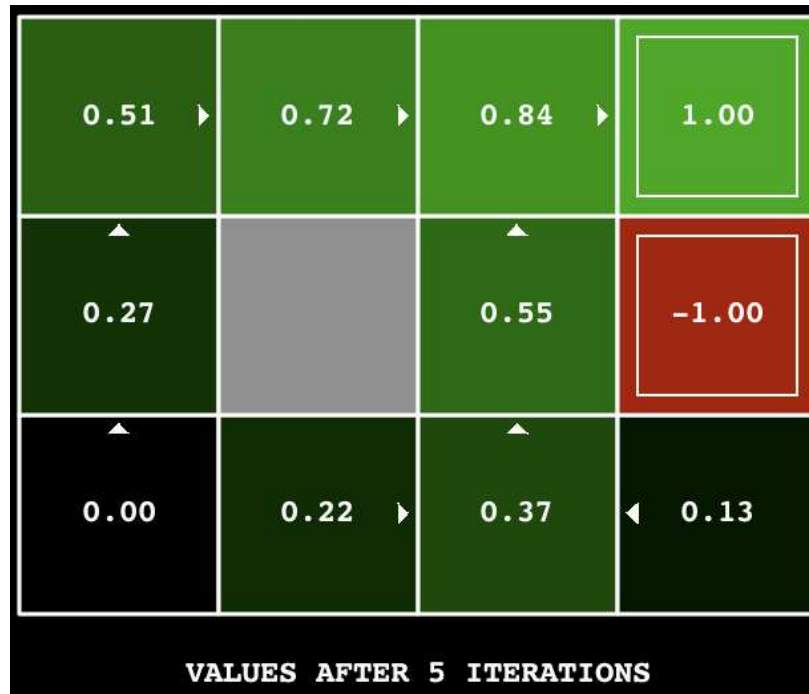
The following command loads your `ValueIterationAgent`, which will compute a policy and execute it 10 times. Press a key to cycle through values, Q-values, and the simulation. You should find that the value of the start state (`V(start)`), which you can read off of the

GUI) and the empirical resulting average reward (printed after the 10 rounds of execution finish) are quite close.

```
python gridworld.py -a value -i 100 -k 10
```

Hint: On the default `BookGrid`, running value iteration for 5 iterations should give you this output:

```
python gridworld.py -a value -i 5
```



Grading: Your value iteration agent will be graded on a new grid. We will check your values, Q-values, and policies after fixed numbers of iterations and at convergence (e.g. after 100 iterations).

Question 2 (6 points): Q-Learning

Note that your value iteration agent does not actually learn from experience. Rather, it ponders its MDP model to arrive at a complete policy before ever interacting with a real environment. When it does interact with the environment, it simply follows the precomputed policy (e.g. it becomes a reflex agent). This distinction may be subtle in a simulated environment like a Gridworld, but it's very important in the real world, where the real MDP is not available.

You will now write a Q-learning agent, which does very little on construction, but instead learns by trial and error from interactions with the environment through its `update(state, action, nextState, reward)` method. A stub of a Q-learner is specified in `QLearningAgent` in `qlearningAgents.py`, and you can select it with the option `'-a q'`. For this question, you must implement the `update`, `computeValueFromQValues`, `getQValue`, and `computeActionFromQValues` methods.

Note: For `computeActionFromQValues`, you should break ties randomly for better behavior. The `random.choice()` function will help.

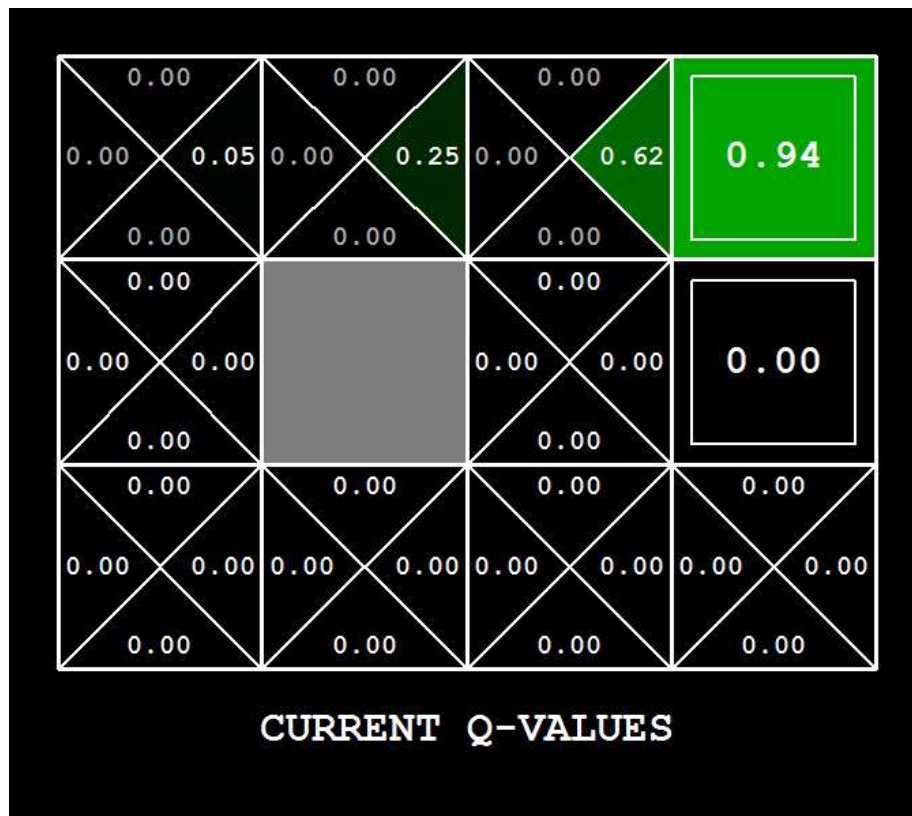
Note: In a particular state, actions that your agent *hasn't* seen before still have a Q-value, specifically a Q-value of zero, and if all of the actions that your agent *has* seen before have a negative Q-value, an unseen action may be optimal.

Important: Make sure that in your `computeValueFromQValues` and `computeActionFromQValues` functions, you only access Q values by calling `getQValue`.

With the Q-learning update in place, you can watch your Q-learner learn under manual control, using the keyboard:

```
python gridworld.py -a q -k 5 -m
```

Recall that `-k` will control the number of episodes your agent gets to learn. Watch how the agent learns about the state it was just in, not the one it moves to, and "leaves learning in its wake." Hint: to help with debugging, you can turn off noise by using the `--noise 0.0` parameter (though this obviously makes Q-learning less interesting). If you manually steer Pacman north and then east along the optimal path for four episodes, you should see the following Q-values:



Grading: We will run your Q-learning agent and check that it learns the same Q-values and policy as our reference implementation when each is presented with the same set of examples.

Question 3 (2 points): Epsilon Greedy

Complete your Q-learning agent by implementing epsilon-greedy action selection in `getAction`, meaning it chooses random actions an epsilon fraction of the time, and follows its current best Q-values otherwise. Note that choosing a random action may result in choosing the best action - that is, you should not choose a random sub-optimal action, but rather any random legal action.

You can choose an element from a list uniformly at random by calling the `random.choice` function. You can simulate a binary variable with probability `p` of success by using `util.flipCoin(p)`, which returns `True` with probability `p` and `False` with probability `1-p`.

After implementing the `getAction` method, observe the following behavior of the agent in `GridWorld` (with `epsilon = 0.3`).

```
python gridworld.py -a q -k 100
```


Your final Q-values should resemble those of your value iteration agent, especially along well-traveled paths. However, your average returns will be lower than the Q-values predict because of the random actions and the initial learning phase.

You can also observe the following simulations for different epsilon values. Does that behavior of the agent match what you expect?

```
python gridworld.py -a q -k 100 --noise 0.0 -e 0.1
```

```
python gridworld.py -a q -k 100 --noise 0.0 -e 0.9
```

With no additional code, you should now be able to run a Q-learning crawler robot:

```
python crawler.py
```

If this doesn't work, you've probably written some code too specific to the `GridWorld` problem and you should make it more general to all MDPs.

This will invoke the crawling robot from class using your Q-learner. Play around with the various learning parameters to see how they affect the agent's policies and actions. Note that the step delay is a parameter of the simulation, whereas the learning rate and epsilon are parameters of your learning algorithm, and the discount factor is a property of the environment.